CIS 3370 · INTRUSION DETECTION SYSTEMS · LAB

Ship logs with Filebeat: a file, a module, and Zeek

Elasticsearch only knows what something sends it. In this lab you install the sender, Filebeat, on two lab machines and point it at three kinds of evidence: a plain file, the system logs of a Linux machine, and the connection records Zeek writes about a network. Each one goes in twice, once as raw lines and once through a module, so you can see exactly what a module adds. Filebeat logs in as a user that can write and do nothing else. Every number on this page was produced by running these steps on the class stack.

1. Look at the cluster
2. Install Filebeat
3. A user that can only write
4. A file as a stream
5. The system module
6. Zeek, stream and module
7. Zeek, live

0 Before you start

You need the class docker-elk stack running, with its five services: `elasticsearch`, `logstash`, `kibana`, `client-01` and `zeek-lab`. The compose file is in Appendix A. If `docker compose ps` in your `docker-elk` folder does not list all five as running, build the stack first.

You also need `cis3370-generate-bundle.zip` from Canvas. It holds the capture and the baselines from Lecture 6, made on our own lab machine. Put a copy in each lab machine's data folder. On a Mac or Linux:

```
cp ~/Downloads/cis3370-generate-bundle.zip ~/docker-elk/client-01/data/
cp ~/Downloads/cis3370-generate-bundle.zip ~/docker-elk/zeek-lab/data/
```

On Windows, in PowerShell:

```
$zip = "$HOME\Downloads\cis3370-generate-bundle.zip"
Copy-Item $zip $HOME\docker-elk\client-01\data\
Copy-Item $zip $HOME\docker-elk\zeek-lab\data\
```

Open two terminals, both in your `docker-elk` folder. One will be inside `client-01` and the other inside `zeek-lab`. Inside a container you are root, the prompt starts in `/home/ubuntu`, and `/home/ubuntu/data` is the folder on your computer.

ACCOUNT	PASSWORD	USED FOR

`elastic`	`changeme`	Setup only: creating the user, the templates and the pipelines, and checking results
`beats`	`cis3370-beats`	Filebeat, every time it sends data

Both passwords are development passwords for a stack on your own laptop. Never use them on anything another machine can reach.

1 Look at the cluster

Before you change anything, ask Docker what is running, which ports each container publishes, and which folders each one mounts. From your `docker-elk` folder on your computer:

```
docker compose ps --format "table {{.Name}}\t{{.Ports}}"
docker compose ps --format "table {{.Name}}\t{{.Mounts}}" --no-trunc
```

The field is `.Name`. If you type `.Names`, which is the field name for `docker ps`, every name prints as `<no value>`. This is what the class stack printed, with the home folder shortened to `you`:

```
NAME                        PORTS
docker-elk-client-01-1
docker-elk-elasticsearch-1  0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp,
0.0.0.0:9300->9300/tcp, [::]:9300->9300/tcp
docker-elk-kibana-1         0.0.0.0:5601->5601/tcp, [::]:5601->5601/tcp
docker-elk-logstash-1       0.0.0.0:5044->5044/tcp, [::]:5044->5044/tcp,
0.0.0.0:9600->9600/tcp, ...
docker-elk-zeek-lab-1

NAME                        <no value>
docker-elk-client-01-1      /host_mnt/Users/you/docker-elk/client-01/data
docker-elk-elasticsearch-1  /host_mnt/Users/you/docker-
elk/elasticsearch/config/elasticsearch.yml,docker-elk_elasticsearch
docker-elk-kibana-1         /host_mnt/Users/you/docker-
elk/kibana/config/kibana.yml
docker-elk-logstash-1       /host_mnt/Users/you/docker-
elk/logstash/config/logstash.yml,/host_mnt/Users/you/docker-
elk/logstash/pipeline
docker-elk-zeek-lab-1       /host_mnt/Users/you/docker-elk/zeek-lab/data
```

Read it row by row:

- **The two lab machines publish no ports.** Nothing on your laptop or your network can reach them. They reach the stack, and each other, over the Docker network by name.
- **Elasticsearch is on 9200 and Kibana on 5601.** Those are the two you use from your browser and from `curl` on your computer.
- **The Mounts column has no heading of its own,** so Compose prints `<no value>` in the header row. The rows are correct.
- **Each lab machine mounts one folder,** its `data` folder. A file you put there from your computer appears at `/home/ubuntu/data` inside it. Elasticsearch keeps its documents in a named volume, `docker-elk_elasticsearch`, which is what `docker compose down -v` deletes.

2 Install Filebeat on client-01

This follows Elastic's page *Filebeat quick start: installation and configuration*, step 1, the DEB tab. Two things are different here. First, the version is pinned to **9.5.3**, the version the cluster runs, because Filebeat should never be newer than Elasticsearch. Second, the download picks your processor type for you. That is `arm64` on an Apple silicon Mac and `amd64` on most Windows and Intel machines.

```
docker compose exec client-01 bash
```

Then, inside `client-01`:

```
apt-get update
apt-get install -y curl unzip nano
cd /tmp
ARCH=$(dpkg --print-architecture)
URL=https://artifacts.elastic.co/downloads/beats/filebeat
curl -L -O $URL/filebeat-9.5.3-${ARCH}.deb
dpkg -i filebeat-9.5.3-${ARCH}.deb
filebeat version
```

Expected output, with your own processor type in the brackets:

```
filebeat version 9.5.3 (arm64), libbeat 9.5.3 [...]
```

The package puts its files in four places, and you will use all of them:

PATH	WHAT IS THERE
`/etc/filebeat/filebeat.yml`	The main configuration: what to read and where to send it

`/etc/filebeat/modules.d/`	One file per module. A file ending in `.disabled` is switched off.
`/var/lib/filebeat/registry`	The registry: how far into each file Filebeat has already read
`/var/log/filebeat/`	Filebeat's own logs, including the record of any event it could not deliver

The Elastic page says `systemctl start filebeat`. That does not work here. A container runs one program, not a full operating system, so it has no `systemd` to start services for you. In this lab you start Filebeat yourself with `nohup filebeat -e` and stop it with `pkill filebeat`.

3 A user that can only write

Filebeat has two jobs, and they need very different rights.

- **Setup** happens once. It creates the index template that says what type each field is, the lifecycle policy that says when old data rolls over, and the ingest pipelines that parse each module's lines. Setup needs administrator rights.
- **Publishing** happens all day, on every machine that ships logs. It only adds documents.

A shipper runs on the machine you are watching. If that machine is broken into, the attacker gets the shipper's password from its configuration file. So the publishing account should be able to add documents to its own data and do nothing else: not read, not delete, not create users. Setup is done once by the `elastic` superuser, and the password Filebeat keeps belongs to a user that cannot undo anything.

PRIVILEGE	WHERE	WHY FILEBEAT NEEDS IT
`monitor`	cluster	Check the Elasticsearch version and license when it connects
`read_ilm`	cluster	Check that the lifecycle policy exists
`read_pipeline`	cluster	Check that a module's ingest pipeline exists before sending data through it
`create_doc`	`filebeat-*`	Add new documents. It cannot change or delete existing ones.

`auto_configure`	`filebeat-*`	Create the data stream on first write, and add a field it has never seen to the mapping

Create the role and the user from `client-01`. The name `elasticsearch` resolves on the Docker network, so `client-01` talks to the cluster directly:

```
ES=http://elasticsearch:9200

curl -u elastic:changeme -X PUT "$ES/_security/role/filebeat_publisher" \
  -H 'Content-Type: application/json' -d '{
  "cluster": ["monitor", "read_ilm", "read_pipeline"],
  "indices": [
    {
      "names": ["filebeat-*"],
      "privileges": ["create_doc", "auto_configure"]
    }
  ]
}'

curl -u elastic:changeme -X PUT "$ES/_security/user/beats" \
  -H 'Content-Type: application/json' -d '{
  "password": "cis3370-beats",
  "roles": ["filebeat_publisher"],
  "full_name": "Filebeat publisher"
}'
```

Expected: `{"role":{"created":true}}`, then `{"created":true}`. Now prove the restriction holds before you rely on it. Each command prints only the HTTP status code:

```
curl -u beats:cis3370-beats "$ES/_security/_authenticate?pretty"
curl -s -o /dev/null -w "%{http_code}\n" -u beats:cis3370-beats \
  "$ES/_cluster/health"
curl -s -o /dev/null -w "%{http_code}\n" -u beats:cis3370-beats \
  "$ES/nsl-kdd-train/_search"
curl -s -o /dev/null -w "%{http_code}\n" -u beats:cis3370-beats \
  "$ES/_security/user"
curl -s -o /dev/null -w "%{http_code}\n" -u beats:cis3370-beats \
  -X POST "$ES/lab-probe/_doc" \
  -H 'Content-Type: application/json' -d '{"x":1}'
```

REQUEST	STATUS	MEANING
Who am I	200	The user exists, and the only role it holds is `filebeat_publisher`
Cluster health	200	`monitor` allows it

Search another index	403	It cannot read anything, not even its own data
List users	403	It cannot see or change accounts
Write outside `filebeat-*`	403	It can write only where its role says

The same role can be built in Kibana under **Stack Management → Security → Roles → Create role**, and the user under **Users**. The API is used here because you can paste it and repeat it exactly.

4 A simple file, read as a stream

Start with the simplest thing Filebeat can do: read a file line by line, send every line, then keep watching the file and send each new line as it is written. The input that does this is called `filestream`. The file is `host-baseline.tsv` from Lecture 6: a header and 60 rows, one per second of a quiet minute on our lab machine.

```
cd /home/ubuntu/data
unzip -o cis3370-generate-bundle.zip
mkdir -p lab
cp cis3370-generate-bundle/host-baseline.tsv lab/
wc -l lab/host-baseline.tsv
```

Expected: `61 lab/host-baseline.tsv`. Keep that number: 61 lines should become 61 documents.

Write the configuration

Keep the original file as a reference, then replace it with one that holds only what this lab uses:

```
cp /etc/filebeat/filebeat.yml /etc/filebeat/filebeat.yml.original

cat > /etc/filebeat/filebeat.yml <<'EOF'
filebeat.inputs:
  - type: filestream
    id: host-baseline
    paths:
      - /home/ubuntu/data/lab/host-baseline.tsv
    fields:
      lab: simple-file

filebeat.config.modules:
  path: ${path.config}/modules.d/*.yml
```

```
  reload.enabled: false

setup.kibana:
  host: "http://kibana:5601"

output.elasticsearch:
  hosts: ["http://elasticsearch:9200"]
  username: "beats"
  password: "cis3370-beats"
EOF
```

SETTING	WHAT IT DOES
`type: filestream`	Read files line by line and keep following them
`id: host-baseline`	A name for this input. Every filestream input needs its own, because the registry tracks progress by it.
`fields.lab`	A label stamped on every document from this input, so you can find them later
`filebeat.config.modules`	Load any module that is switched on in `modules.d/`. None is yet.
`output.elasticsearch`	Where to send the data, and as which user

Test it before you run it

```
filebeat test config
filebeat test output
```

Expected: `Config OK`, then a connection check that ends:

```
  talk to server... OK
  version: 9.5.3
```

A `WARN secure connection disabled` line in between is correct. This stack speaks plain HTTP, which is fine while nothing leaves your laptop.

Run setup, first as the wrong user

Run setup as `beats`, the user in the file, and watch it fail:

```
filebeat setup --index-management
```

```
Exiting: ... got 403 from elasticsearch: ... action [cluster:admin/ilm/put]
is unauthorized
for user [beats] with effective roles [filebeat_publisher], this action is
granted by the
cluster privileges [manage_ilm,manage,all]
```

That refusal is the role working. Now run the same command as `elastic`. The `-E` flags replace the username and password for this one command only, and leave the file alone:

```
filebeat setup --index-management \
  -E output.elasticsearch.username=elastic \
  -E output.elasticsearch.password=changeme
```

Expected: `Index setup finished.`

Start Filebeat and count

```
nohup filebeat -e > /var/log/filebeat-run.log 2>&1 &
```

Filebeat now runs in the background and writes its own log to `/var/log/filebeat-run.log`. `nohup` keeps it running after you close the terminal. Wait about 20 seconds, then count. Use `elastic` for this, because `beats` cannot read:

```
curl -s -u elastic:changeme "$ES/filebeat-*/_count?q=fields.lab:simple-file"
```

```
{"count":61,"_shards":{"total":1,"successful":1,"skipped":0,"failed":0}}
```

61 lines in the file, 61 documents in Elasticsearch. Now the streaming part. Add one line to the file while Filebeat watches it:

```
echo "client-01 appended this line" >> lab/host-baseline.tsv
```

Filebeat looks for changes every 10 seconds. Count again after about 20 seconds and you get `"count":62`. Nobody restarted anything; the new line was picked up because Filebeat remembers where it stopped reading. That memory is the registry.

If the count stays at 0. When this lab was first written, the role had `create_doc` and not `auto_configure`. Filebeat connected, read all 61 lines, and dropped every one. Its log said only this:

```
Failed to index 61 events in last 10s: events were dropped! Check the
event_data log ...
```

The reason was in `/var/log/filebeat/filebeat-events-data-*.ndjson`, the file Filebeat keeps for events it could not deliver:

```
(status=403): action [indices:admin/mapping/auto_put] is unauthorized for
user [beats]
```

The field `fields.lab` was new to Elasticsearch, and adding a new field to the mapping needs `auto_configure`. If this happens to you, fix the role, then make Filebeat forget it already read the file, or it will never send those lines again:

```
pkill filebeat
rm -rf /var/lib/filebeat/registry
nohup filebeat -e > /var/log/filebeat-run.log 2>&1 &
```

See it in Kibana

1. Open http://localhost:5601 and log in as `elastic`.
2. **Stack Management → Data Views → Create data view.** Name it `filebeat`, set the index pattern to `filebeat-*` and the timestamp field to `@timestamp`, then save.
3. Open **Discover**, pick `filebeat`, set the time to **Last 1 hour**, and search `fields.lab : "simple-file"`.

Each document's `message` is one whole line of the file, tabs and all. Filebeat did not split it into columns, and its `@timestamp` is the moment Filebeat read the line, not the time written in the line. Remember both facts for section 6.

5 The system module

A module is a packaged answer to the question "how do I read this kind of log?" It knows where the files usually live, and it ships an ingest pipeline that splits each line into named fields: `user.name`, `source.ip`, `event.outcome`. The system module reads the two logs every Linux machine keeps: `syslog`, for general messages, and `auth.log`, for logins.

Give client-01 something to log

A container has no logging service and no SSH server, so the two files do not exist yet. Install both, and start them by hand. On `client-01`:

```
apt-get install -y rsyslog openssh-server
sed -i 's/^module(load="imklog"/#&/' /etc/rsyslog.conf
rsyslogd
mkdir -p /run/sshd && /usr/sbin/sshd
ls -l /var/log/syslog /var/log/auth.log
```

The `sed` line switches off the part of rsyslog that reads kernel messages, because a container is not allowed to read the kernel's log. Both files should now exist. If `ls` says `No such file or directory`, rsyslog has not finished starting: wait a second and run the `ls` line again.

Enable the module, then configure it

```
filebeat modules enable system
```

Expected: `Enabled system`. That renamed `modules.d/system.yml.disabled` to `system.yml`. Inside it, both parts of the module, called *filesets*, are still switched off. Switch them on and say where the files are:

```
cat > /etc/filebeat/modules.d/system.yml <<'EOF'
- module: system
  syslog:
    enabled: true
    var.paths: ["/var/log/syslog"]
  auth:
    enabled: true
    var.paths: ["/var/log/auth.log"]
EOF
```

Test the module configuration

```
filebeat test config
filebeat modules list | head -4
```

```
Config OK
Enabled:
system

Disabled:
```

Load the module's pipelines, as elastic

The pipelines are part of setup, so `beats` cannot load them. Load them as `elastic`:

```
filebeat setup --pipelines --modules system --force-enable-module-filesets \
  -E output.elasticsearch.username=elastic \
  -E output.elasticsearch.password=changeme
```

Expected: `Loaded Ingest pipelines`. Without `--force-enable-module-filesets` you get `Exiting: module system is configured but has no enabled filesets`. That is because with `--modules`, setup ignores the file you just wrote and looks only at the command line.

Restart Filebeat so it loads the module:

```
pkill filebeat
nohup filebeat -e > /var/log/filebeat-run.log 2>&1 &
```

If you started Filebeat before loading the pipelines, its log repeats `Failed to connect to backoff(...)` together with `cluster:admin/ingest/pipeline/put is unauthorized for user [beats]`. Filebeat tried to install a missing pipeline itself and was refused. Load the pipelines as above. Filebeat keeps retrying and connects on its own once they exist.

Make some evidence

Write one line to syslog on `client-01`:

```
logger -t cis3370 "hello from client-01 through syslog"
```

Then try to log in to `client-01` from the other machine, three times, as three different users. In your second terminal:

```
docker compose exec zeek-lab bash
```

```
apt-get update
apt-get install -y openssh-client
for u in admin root test; do
  ssh -o BatchMode=yes -o StrictHostKeyChecking=no -o ConnectTimeout=5 \
    $u@client-01 true
done
```

Each attempt prints `Permission denied (publickey,password).` Back on `client-01`, `tail -5 /var/log/auth.log` shows what the server wrote about it:

```
... sshd-session[3438]: Invalid user admin from 172.21.0.8 port 40830
... sshd-session[3438]: Connection closed by invalid user admin 172.21.0.8
port 40830 [preauth]
... sshd-session[3440]: Connection closed by authenticating user root
172.21.0.8 port 40844 [preauth]
... sshd-session[3442]: Invalid user test from 172.21.0.8 port 40860
... sshd-session[3442]: Connection closed by invalid user test 172.21.0.8
port 40860 [preauth]
```

The address is `zeek-lab`'s on the class stack. Yours will differ.

Validate that the data is in Elasticsearch

Count documents by where they came from. The field `event.dataset` names the module and fileset:

```
curl -s -u elastic:changeme "$ES/filebeat-*/_search?size=0&pretty" \
  -H 'Content-Type: application/json' \
  -d '{"aggs":{"datasets":{"terms":{"field":"event.dataset"}}}}'
```

You should see buckets for `system.auth` and `system.syslog`. The 62 documents from section 4 have no dataset, so they are not in the list. In `system.auth`, starting sshd adds 2 documents, and each round of three login attempts adds 5. Then ask for the failed logins only:

```
curl -s -u elastic:changeme -G "$ES/filebeat-*/_search" -d pretty \
  -d q=system.auth.ssh.event:Invalid \
  -d _source=user.name,source.ip,event.outcome
```

Each hit now has fields the raw line did not: `user.name` is `admin` or `test`, `source.ip` is `zeek-lab`'s address, and `event.outcome` is `failure`. The pipeline handled the newer OpenSSH name for the login process, `sshd-session`, without any change. In Discover the same search is `event.dataset : "system.auth" and event.outcome : "failure"`.

6 Zeek on zeek-lab: a stream, then a module

Now the same exercise on the network side. Zeek reads packets and writes one log line per connection, DNS query and TLS handshake. You will run Zeek on the Lecture 6 capture, ship its `conn.log` as raw lines, ship the same logs through Filebeat's Zeek module, and compare the two.

Install Zeek

Ubuntu has no Zeek package of its own. The Zeek project publishes one for Ubuntu 26.04, for both processor types. In your `zeek-lab` terminal:

```
apt-get install -y curl gnupg ca-certificates unzip nano
REPO=download.opensuse.org/repositories/security:/zeek/xUbuntu_26.04
echo "deb http://$REPO/ /" > /etc/apt/sources.list.d/zeek.list
curl -fsSL https://$REPO/Release.key \
  | gpg --dearmor > /etc/apt/trusted.gpg.d/zeek.gpg
apt-get update
apt-get install -y zeek-core
/opt/zeek/bin/zeek --version
```

Expected: `/opt/zeek/bin/zeek version 9.0.1`, or a later 9.0 release. `zeek-core` is the analysis engine alone, which is all this lab needs.

Install Filebeat on zeek-lab

Run the same seven commands as in section 2, starting at `apt-get update`. The user and the role already exist, because they live in Elasticsearch and not on a machine.

Turn the capture into Zeek logs

```
cd /home/ubuntu/data
unzip -o cis3370-generate-bundle.zip
mkdir -p zeek-logs && cd zeek-logs
/opt/zeek/bin/zeek -C -r ../cis3370-generate-bundle/capture.pcap \
  LogAscii::use_json=T
wc -l *.log
```

```
  2024 conn.log
    16 dns.log
     1 packet_filter.log
     8 ssl.log
```

PART	MEANING
`-r capture.pcap`	Read packets from a file instead of a network interface
`-C`	Ignore checksums. A container's own outgoing packets are captured before the network card fills in the checksum, so without this flag Zeek throws them away as damaged.
`LogAscii::use_json=T`	Write each record as one JSON object instead of tab-separated text. Filebeat's Zeek module reads JSON only.

Look at one record, and note its `uid`. You will look this connection up twice:

```
head -1 conn.log
```

```
{"ts":1793423385.033747,"uid":"CIUp8t36K7cTo9XKd3","id.orig_h":"172.21.0.5","id.
"id.resp_h":"216.59.63.135","id.resp_p":443,"proto":"tcp","service":"ssl",
... }
```

Zeek makes up a new `uid` every time it runs, so yours is different. The other values are the same, because they come from the packets.

Ship conn.log as a stream

This is section 4 again, with one addition. Each line is JSON, so a parser called `ndjson` turns the line into fields, and `target` puts them under one name so they cannot collide with anything else:

```
cp /etc/filebeat/filebeat.yml /etc/filebeat/filebeat.yml.original

cat > /etc/filebeat/filebeat.yml <<'EOF'
filebeat.inputs:
  - type: filestream
    id: zeek-conn-raw
    paths:
      - /home/ubuntu/data/zeek-logs/conn.log
    parsers:
      - ndjson:
          target: "zeek_raw"
    fields:
      lab: zeek-stream

filebeat.config.modules:
  path: ${path.config}/modules.d/*.yml
  reload.enabled: false

setup.kibana:
  host: "http://kibana:5601"

output.elasticsearch:
  hosts: ["http://elasticsearch:9200"]
  username: "beats"
  password: "cis3370-beats"
EOF
```

Enable and configure the Zeek module

```
filebeat modules enable zeek
nano /etc/filebeat/modules.d/zeek.yml
```

The file lists 39 filesets, one per Zeek log, all switched off. Change three of them, and write the paths exactly as shown. Each fileset reads the bundle's logs now, and the live logs of section 7 later:

```
  connection:
    enabled: true
    var.paths:
      - /home/ubuntu/data/zeek-logs/conn.log
      - /home/ubuntu/data/zeek-live/conn.log
  ...
  dns:
    enabled: true
    var.paths:
      - /home/ubuntu/data/zeek-logs/dns.log
      - /home/ubuntu/data/zeek-live/dns.log
  ...
  ssl:
    enabled: true
    var.paths:
      - /home/ubuntu/data/zeek-logs/ssl.log
      - /home/ubuntu/data/zeek-live/ssl.log
```

Save with **Ctrl+O**, then **Enter**, and exit with **Ctrl+X**. The Zeek file is called `conn.log`, but the fileset is called `connection`. Check your edit, then test:

```
grep -A4 -E "^  (connection|dns|ssl):" /etc/filebeat/modules.d/zeek.yml
filebeat test config
filebeat test output
```

Load the template and the Zeek pipelines as `elastic`, then start Filebeat:

```
filebeat setup --index-management --pipelines \
  --modules zeek --force-enable-module-filesets \
  -E output.elasticsearch.username=elastic \
  -E output.elasticsearch.password=changeme
nohup filebeat -e > /var/log/filebeat-run.log 2>&1 &
```

Expected from setup: `Index setup finished.` and `Loaded Ingest pipelines`.

Validate: every line arrived, twice

```
ES=http://elasticsearch:9200
curl -s -u elastic:changeme "$ES/filebeat-*/_count?q=fields.lab:zeek-stream"
curl -s -u elastic:changeme "$ES/filebeat-*/_search?size=0&pretty" \
  -H 'Content-Type: application/json' \
  -d '{"aggs":{"datasets":{"terms":{"field":"event.dataset"}}}}'
```

WHAT	LINES ZEEK WROTE	DOCUMENTS IN ELASTICSEARCH
Raw stream of `conn.log`	2,024	2,024
`zeek.connection`	2,024	2,024
`zeek.dns`	16	16
`zeek.ssl`	8	8

Check every count against `wc -l`. A missing line is a lost connection, and an IDS that quietly loses records is worse than one that is down, because nobody knows to look.

Compare the same connection, both ways

Put your own `uid` on the first line:

```
ZUID=CIUp8t36K7cTo9XKd3
curl -s -u elastic:changeme -G "$ES/filebeat-*/_search" -d pretty \
  -d q=zeek_raw.uid:$ZUID -d _source=@timestamp,zeek_raw
curl -s -u elastic:changeme -G "$ES/filebeat-*/_search" -d pretty \
  -d q=zeek.session_id:$ZUID \
  -d _source=@timestamp,source,destination,network,zeek.connection
curl -s -u elastic:changeme -G -d pretty \
  "$ES/filebeat-*/_mapping/field/zeek_raw.ts,zeek_raw.id.orig_h,source.ip"
```

QUESTION	RAW STREAM	ZEEK MODULE
When did it happen?	`@timestamp` is when Filebeat read the line. The real time is `zeek_raw.ts`, a plain number of seconds.	`@timestamp` is the connection's start, taken from `ts`
Who connected?	`zeek_raw.id.orig_h`, stored as `keyword`, which is text	`source.ip`, stored as `ip`, so ranges such as `172.21.0.0/16` work
Field names	Zeek's own: `id.resp_p`, `conn_state`	ECS, the names every Elastic tool expects: `destination.port`, `zeek.connection.state`

Anything added?	No	Country and network owner of the outside address (`destination.geo`, `destination.as`), a `network.community_id`, and `state_message: "Normal establishment and termination."`

The module gives you nothing Zeek did not already know. It gives you the same facts under the names and types that searching, dashboards and detection rules are built on.

Find the scan

The bundle's capture holds the minute in which `nmap -sT` scanned 1,000 ports on each of the machine's two addresses. Ask Elasticsearch how the connections ended:

```
curl -s -u elastic:changeme "$ES/filebeat-*/_search?size=0&pretty" \
  -H 'Content-Type: application/json' -d '{
  "query": {"term": {"event.dataset": "zeek.connection"}},
  "aggs": {
    "state": {"terms": {"field": "zeek.connection.state"}},
    "span":  {"stats": {"field": "@timestamp"}},
    "rejected": {
      "filter": {"term": {"zeek.connection.state": "REJ"}},
      "aggs": {
        "ports":   {"cardinality": {"field": "destination.port"}},
        "targets": {"terms": {"field": "destination.ip"}}
      }
    }
  }
}'
```

CONNECTION STATE	MEANING	CONNECTIONS
`REJ`	Asked to connect and was refused straight away	2,000
`SF`	Opened and closed normally	8
`S0`	Asked to connect and never got an answer	8
`SHR`	Zeek saw only the reply side	8

The 2,000 refused connections hit **1,000 distinct ports**: 1,000 on `127.0.0.1` and 1,000 on `172.21.0.5`, the two addresses the scan named. Everything else in the capture is 24 connections: the eight page loads and the DNS lookups behind them. The whole capture spans

43.7 seconds, from `05:09:45.028` to `05:10:28.722` UTC. To see it in Discover, set an absolute time range from the `span` minimum to its maximum.

7 Zeek, live

So far Zeek read a recording. Now it watches `zeek-lab`'s own network card while `client-01` scans it, and Filebeat ships each record as Zeek writes it. The module already lists the `zeek-live` folder, so nothing in Filebeat changes. When the file appears, Filebeat finds it within about 10 seconds. On `zeek-lab`:

```
mkdir -p /home/ubuntu/data/zeek-live && cd /home/ubuntu/data/zeek-live
/opt/zeek/bin/zeek -C -i eth0 LogAscii::use_json=T > zeek.out 2>&1 &
hostname -i
```

`-i eth0` means listen on the network card. Note the address `hostname -i` prints. On `client-01`:

```
apt-get install -y nmap
nmap -sT -p 1-1000 zeek-lab
hostname -i
```

```
Not shown: 1000 closed tcp ports (conn-refused)
...
Nmap done: 1 IP address (1 host up) scanned in 0.60 seconds
```

Back on `zeek-lab`, stop Zeek politely so it writes out everything it is holding, then count:

```
pkill -INT zeek
wc -l conn.log
cat reporter.log
```

On the class stack: `1001 conn.log`, and the reporter log said `2006 packets received on interface eth0, 0 (0.00%) dropped`. A capture that drops packets misses connections, so check that number every time you capture. Now ask Elasticsearch about the live file only:

```
curl -s -u elastic:changeme "$ES/filebeat-*/_search?size=0&pretty" \
  -H 'Content-Type: application/json' -d '{
  "query": {"bool": {"filter": [
    {"term": {"event.dataset": "zeek.connection"}},
    {"term": {"log.file.path": "/home/ubuntu/data/zeek-live/conn.log"}}
  ]}},
```

```
  "aggs": {
    "from":  {"terms": {"field": "source.ip"}},
    "state": {"terms": {"field": "zeek.connection.state"}},
    "ports": {"cardinality": {"field": "destination.port"}}
  }
}'
```

On the class stack: 1,001 documents. 1,000 came from `client-01`'s address, all `REJ`, and one record was `zeek-lab`'s own traffic. That is the scan, seen from the target, a few seconds after it happened. The bundle counts do not change: Filebeat's registry remembers it already sent those files, so restarting it never sends them twice.

8 Take the password out of the file

`filebeat.yml` still holds the `beats` password in plain text. Filebeat has a keystore for exactly this. Do it on both machines:

```
filebeat keystore create
echo -n "cis3370-beats" | filebeat keystore add BEATS_PWD --stdin
filebeat keystore list
sed -i 's|password: "cis3370-beats"|password: "${BEATS_PWD}"|' \
  /etc/filebeat/filebeat.yml
grep password /etc/filebeat/filebeat.yml
filebeat test output
pkill filebeat
nohup filebeat -e > /var/log/filebeat-run.log 2>&1 &
```

Expected: the list prints `BEATS_PWD`, the file now says `password: "${BEATS_PWD}"`, and the output test still ends with `talk to server... OK`. A line sent after the restart arrived as normal. The password is still on the machine, because Filebeat has to send it. The keystore keeps it out of a file people read, copy and paste into tickets.

9 Questions

QUESTION 1

Setup was refused for `beats` and worked for `elastic`. Name the privilege each refusal said was missing, and say why a shipper should never hold it.

QUESTION 2

`beats` cannot read even its own data. An attacker owns `client-01` and has its password. List what they can do to your Elasticsearch, and what they cannot.

QUESTION 3

The same connection has two different `@timestamp` values in section 6. A detection rule counts refused connections per minute. What would it report on the raw stream, and why is that dangerous?

QUESTION 4

In `auth.log`, `admin` and `test` are "Invalid user" but `root` is "authenticating user". What did the server just tell the person trying to log in, and what does it tell you?

QUESTION 5

The bundle shows 2,000 refused connections and the live scan shows 1,000, from the same kind of nmap command. Explain the difference using the targets each scan named.

QUESTION 6

Choose a threshold: how many `REJ` connections from one source within 60 seconds should raise an alert? Use the quiet traffic in the same capture, 24 connections, to justify it, and say what kind of scan your threshold would miss.

QUESTION 7

`client-01` ran the live scan. What did `client-01`'s own `syslog` and `auth.log` record about it? What does that say about relying on host logs alone?

10 What to hand in

1. The output of the two `docker compose ps` commands from section 1.
2. The status codes from the five checks in section 3.
3. One table with your counts: lines in each file, and documents in Elasticsearch, for the simple file before and after the append, `system.auth`, `system.syslog`, the raw Zeek stream, the three Zeek datasets, and the live capture.
4. A Discover screenshot of the failed logins from section 5, and one of the refused connections from section 7, each with the search visible.

5. Your answers to the seven questions, one paragraph each, each with its evidence.

11 Start over

To repeat the lab from a clean cluster, stop Filebeat and clear its registry on both machines:

```
pkill filebeat
rm -rf /var/lib/filebeat/registry
```

Then remove what the lab created in Elasticsearch, from either machine:

```
ES=http://elasticsearch:9200
curl -u elastic:changeme -X DELETE "$ES/_data_stream/filebeat-9.5.3"
curl -u elastic:changeme -X DELETE "$ES/_index_template/filebeat-9.5.3"
curl -u elastic:changeme -X DELETE "$ES/_ilm/policy/filebeat"
curl -u elastic:changeme -X DELETE "$ES/_ingest/pipeline/filebeat-9.5.3-*"
curl -u elastic:changeme -X DELETE "$ES/_security/user/beats"
curl -u elastic:changeme -X DELETE "$ES/_security/role/filebeat_publisher"
```

The first command deletes every document Filebeat sent. Nothing else in your cluster is touched.

Going further, not run in this guide

The log bundle from Canvas holds Zeek logs in JSON and Suricata alerts in `eve.json`. Point the Zeek module's `var.paths` at `zeek_json/`, or enable the `suricata` module and point its `eve` fileset at `suricata/eve.json`. This guide did not run either, so report what happened, what you had to change, and whether every line arrived.

A The class docker-elk compose file

This replaces the `docker-compose.yml` of a docker-elk checkout at commit `a41b72b`, the 9.5.3 release. Clone with `git clone https://github.com/deviantony/docker-elk.git`, run `git checkout a41b72b`, put this file in place, and run `mkdir -p zeek-lab/data client-01/data` before the first `docker compose up`.

```
# The class docker-elk environment in one file.
#
# This replaces the docker-compose.yml of a docker-elk checkout at the
# "Update to v9.5.3" commit (a41b72b). It is docker-elk's own file with two
# changes: more memory for Elasticsearch and Logstash, and two Ubuntu lab
```

```
# containers, zeek-lab and client-01, on the same network as the stack.
# The README.md beside this file has the steps.

services:

  # The 'setup' service runs a one-off script which initializes users inside
  # Elasticsearch, such as 'logstash_internal' and 'kibana_system', with the
  # values of the passwords defined in the '.env' file. It also creates the
  # roles required by some of these users.
  #
  # This task only needs to be performed once, during the *initial* startup
of
  # the stack. Any subsequent run will reset the passwords of existing users
to
  # the values defined inside the '.env' file, and the built-in roles to
their
  # default permissions.
  #
  # By default, it is excluded from the services started by 'docker compose
up'
  # due to the non-default profile it belongs to. To run it, either provide
the
  # '--profile=setup' CLI flag to Compose commands, or "up" the service by
name
  # such as 'docker compose up setup'.
  setup:
    profiles:
      - setup
    build:
      context: setup/
      args:
        ELASTIC_VERSION: ${ELASTIC_VERSION}
    init: true
    volumes:
      - ./setup/entrypoint.sh:/entrypoint.sh:ro,Z
      - ./setup/lib.sh:/lib.sh:ro,Z
      - ./setup/roles:/roles:ro,Z
    environment:
      ELASTIC_PASSWORD: ${ELASTIC_PASSWORD:-}
      LOGSTASH_INTERNAL_PASSWORD: ${LOGSTASH_INTERNAL_PASSWORD:-}
      KIBANA_SYSTEM_PASSWORD: ${KIBANA_SYSTEM_PASSWORD:-}
      METRICBEAT_INTERNAL_PASSWORD: ${METRICBEAT_INTERNAL_PASSWORD:-}
      FILEBEAT_INTERNAL_PASSWORD: ${FILEBEAT_INTERNAL_PASSWORD:-}
      HEARTBEAT_INTERNAL_PASSWORD: ${HEARTBEAT_INTERNAL_PASSWORD:-}
      MONITORING_INTERNAL_PASSWORD: ${MONITORING_INTERNAL_PASSWORD:-}
      BEATS_SYSTEM_PASSWORD: ${BEATS_SYSTEM_PASSWORD:-}
    network_mode: service:elasticsearch
    depends_on:
      - elasticsearch
```

```
  # The 'kibana-genkeys' service runs a one-off command which generates and
  # prints encryption keys for Kibana.
  #
  # By default, it is excluded from the services started by 'docker compose
up'
  # due to the non-default profile it belongs to. To run it, either provide
the
  # '--profile=setup' CLI flag to Compose commands, or "up" the service by
name
  # such as 'docker compose up kibana-genkeys'.
  kibana-genkeys:
    profiles:
      - setup
    build:
      context: kibana/
      args:
        ELASTIC_VERSION: ${ELASTIC_VERSION}
    command:
      - bin/kibana-encryption-keys
      - generate
    network_mode: none

  elasticsearch:
    build:
      context: elasticsearch/
      args:
        ELASTIC_VERSION: ${ELASTIC_VERSION}
    volumes:
      -
./elasticsearch/config/elasticsearch.yml:/usr/share/elasticsearch/config/elastic
      - elasticsearch:/usr/share/elasticsearch/data:Z
    ports:
      - 9200:9200
      - 9300:9300
    environment:
      node.name: elasticsearch
      ES_JAVA_OPTS: -Xms2048m -Xmx2048m
      # Bootstrap password.
      # Used to initialize the keystore during the initial startup of
      # Elasticsearch. Ignored on subsequent runs.
      ELASTIC_PASSWORD: ${ELASTIC_PASSWORD:-}
      # Use single node discovery in order to disable production mode and
avoid bootstrap checks.
      # see: https://www.elastic.co/docs/deploy-manage/deploy/self-
managed/bootstrap-checks
      discovery.type: single-node
    networks:
      - elk
    restart: unless-stopped
```

```
  logstash:
    build:
      context: logstash/
      args:
        ELASTIC_VERSION: ${ELASTIC_VERSION}
    volumes:
      -
./logstash/config/logstash.yml:/usr/share/logstash/config/logstash.yml:ro,Z
      - ./logstash/pipeline:/usr/share/logstash/pipeline:ro,Z
    ports:
      - 5044:5044
      - 50000:50000/tcp
      - 50000:50000/udp
      - 9600:9600
    environment:
      LS_JAVA_OPTS: -Xms1024m -Xmx1024m
      LOGSTASH_INTERNAL_PASSWORD: ${LOGSTASH_INTERNAL_PASSWORD:-}
    networks:
      - elk
    depends_on:
      - elasticsearch
    restart: unless-stopped

  kibana:
    build:
      context: kibana/
      args:
        ELASTIC_VERSION: ${ELASTIC_VERSION}
    volumes:
      - ./kibana/config/kibana.yml:/usr/share/kibana/config/kibana.yml:ro,Z
    ports:
      - 5601:5601
    environment:
      KIBANA_SYSTEM_PASSWORD: ${KIBANA_SYSTEM_PASSWORD:-}
    networks:
      - elk
    depends_on:
      - elasticsearch
    restart: unless-stopped

  # Two lab machines on the same 'elk' network as the stack. Both are plain
  # Ubuntu 26.04 that stay running so you can open a shell in them:
  #
  #   docker compose exec zeek-lab bash
  #   docker compose exec client-01 bash
  #
  # Each one mounts a folder from this repository at /home/ubuntu/data, so
files
  # you put there from your computer appear inside the container, and files
the
```

```
  # container writes there (captures, logs) appear on your computer. They
also
  # survive 'docker compose down -v', because they are folders, not volumes.
  #
  # The other containers reach them by name: 'zeek-lab', 'client-01',
  # 'elasticsearch', 'logstash', 'kibana'. NET_ADMIN and NET_RAW let tcpdump
  # capture and nmap send raw packets.
  zeek-lab:
    image: ubuntu:26.04
    hostname: zeek-lab
    command: sleep infinity
    init: true
    working_dir: /home/ubuntu
    volumes:
      - ./zeek-lab/data:/home/ubuntu/data:Z
    cap_add:
      - NET_ADMIN
      - NET_RAW
    networks:
      - elk
    restart: unless-stopped

  client-01:
    image: ubuntu:26.04
    hostname: client-01
    command: sleep infinity
    init: true
    working_dir: /home/ubuntu
    volumes:
      - ./client-01/data:/home/ubuntu/data:Z
    cap_add:
      - NET_ADMIN
      - NET_RAW
    networks:
      - elk
    restart: unless-stopped

networks:
  elk:
    driver: bridge

volumes:
  elasticsearch:
```

Every number on this page was produced by running these steps on the class stack: Filebeat 9.5.3, Elasticsearch 9.5.3, Zeek 9.0.1, Ubuntu 26.04. Addresses, process numbers and Zeek's `uid` values will

differ on your machine. Counts from the bundle should not. If one does, say by how much and find out why.